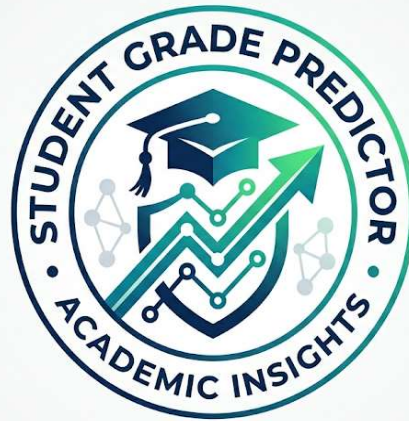




STUDENT GRADE PREDICTOR

STUDENT GRADE PREDICTOR

Name – Sanidhya Moondra

Registration Number – 25BAI10230

Course – Fundamentals of AI and ML

Course Code – CSA2001

Project – Student Grade Predictor

1. Define the Problem

1.1] Context and Motivation

In educational institutions identifying students who are at risk of underperforming is a critical challenge. Educators often rely on delayed indicators such as mid-term exams or final grades, to assess student understanding. By the time these indicators are available, it is frequently too late to intervene effectively.

1.2] The Objective

The core objective of this project is to build a Machine Learning model that can predict a student's final grade (A, B, C, D, or F) based on their daily habits and academic behaviors. By inputting study hours, attendance and sleep schedules the system provides a projected grade.

Furthermore the problem definition extends beyond just prediction. The model is designed to not only predict the outcome but also offer customized, actionable feedback to the user, identifying specific areas for improvement .

1.3] Scope

This project approaches the problem using a supervised machine learning classification framework. Instead of using pre-built libraries the underlying classification algorithm is constructed entirely from scratch using fundamental mathematical principles in Python.

2. Gather Data

2.1] Data Generation Strategy

In many real-world beginner scenarios, data is downloaded from repositories like Kaggle. However, obtaining highly specific, clean and balanced data regarding student personal habits can be difficult due to

privacy concerns. Therefore, this project utilizes a synthetic data generation approach to build a dataset of 200 student records.

2.2] The Mechanism

The generate data(n=200) function simulates a diverse student population. It assigns random values to various features within realistic bounds:

- **Study Hours:** Uniformly distributed between 1 and 10 hours.
- **Attendance:** Uniformly distributed between 40% and 100%.
- **Previous Score:** Uniformly distributed between 30 and 100.
- **Assignments:** Integer values between 0 and 10.
- **Sleep Hours:** Uniformly distributed between 4 and 9 hours.

3. Clean and Prepare Data

3.1] Data Leakage Prevention

Data preparation is the most critical step in this pipeline. In machine learning the model must be evaluated on unseen data. If the model is exposed to the testing data during the preparation phase it results in data leakage, leading to falsely inflated accuracy metrics.

To combat this, the dataset is first divided using the train_test_split function. An 80/20 split is applied: 80% (160 records) are reserved for training and 20% (40 records) are isolated for testing.

3.2] Feature Scaling: Min-Max Normalization

As the chosen algorithm relies on calculating distances between data points and features with larger numerical ranges can unfairly dominate the model. For instance the distance between two attendance values (ranging from 40 to 100) would mathematically overwhelm the distance between two assignments values (ranging from 0 to 10). To solve this, Min-Max

Normalization is applied. The formula scales every numeric value to a proportion between 0 and 1.

4. Explore and Analyze Data

4.1] Distribution Analysis

While the data is generated synthetically the underlying distributions follow logical patterns defined by the generation parameters. Because `random.uniform` and `random.randint`, meaning there is an equal representation of students with high, medium and low study hours.

4.2] Correlation Assumptions

The exploratory phase is built into the logic of the data generation itself. We can assert strong positive linear correlations between study hours, assignments, and the final grade due to the assigned weights in the generation formula. Sleep hours and attendance also share positive correlations with the target variable though with slightly lesser mathematical influence.

The final target variable is a categorized tier (A, B, C, D, F) the dataset presents a multi-class classification problem. The uniform distribution of the input features naturally translates to a bell-curve-like distribution of grades, with a higher concentration of students in the middle grades (B, C) and fewer at the extreme ends (A and F).

5. Select and Create Features

5.1] Feature Selection

The features selected for this model represent a holistic view of a student's academic life, combining effort, history and physical well-being:

1. **Study hours** - Represents direct academic effort.

2. **Attendance** - Represents engagement and exposure to class materials.
3. **Prev score** - Represents the old or previous scores
4. **Assignments** - Represents task completion and discipline.
5. **Sleep hours** - Represents physical readiness

5.2] Target Mapping

The target feature is grade. During the generation phase, continuous numerical scores were discretized into strict categorical bins:

- 1] Score ≥ 75 : Class A
- 2] Score ≥ 60 : Class B
- 3] Score ≥ 45 : Class C
- 4] Score ≥ 30 : Class D
- 5] Score < 30 : Class F

By converting a continuous score into discrete categories, the problem is definitively framed for a classification algorithm rather than a regression model.

6. Choose Algorithm

6.1] K-Nearest Neighbors (KNN)

The algorithm chosen for this project is K-Nearest Neighbors (KNN). KNN is an instance-based, lazy learning algorithm. It does not try to map an internal mathematical function to the data instead it stores the entire training dataset and makes predictions by comparing new data points to the most similar existing data points.

6.2] Mathematical Mechanics

To find the nearest neighbors, the model must calculate the distance between a new student's data profile and every profile in the training set. This implementation utilizes the Euclidean Distance formula across dimensions .

The formula implemented in the euclidean function is:

$$d(x,y) = (x_i - y_i)^2$$

Once the distances to all 160 training samples are calculated, the model sorts them in ascending order.

7. Train the Model

7.1] Instance-Based Training

In traditional parametric models like Linear Regression , the "training" phase involves feeding data through an algorithm to adjust weights and biases until an optimal state is reached after which the original data can be discarded.

KNN operates differently because it is a non-parametric, lazy learner, there is no explicit training phase where weights are adjusted. Instead, "training" the model in this project consists of:

1. Receiving the raw training subset.
2. Calculating the minimum and maximum values strictly from this subset.
3. Mapping the raw training data into the normalized dimensional space.
4. Storing this normalized dataset in the program's memory.

The train data list successfully prepared in the main function essentially is the trained model.

8. Test and Validate Model

Before the model can be trusted with user data, its accuracy must be quantified. This is executed using the evaluate function. The process involves taking the 40 isolated records from the testing split. Crucially, these records are normalized using the boundaries established by the training set, simulating how unseen real-world data would behave.

9. Deploy Model

9.1] Application Interface

Model deployment is handled via a clean Command-Line Interface (CLI). The application is cased within a continuous while loop allowing it to serve multiple users in a single session without requiring a complete restart .

9.2] Inference Pipeline

When a user interacts with the system via the predict_for_student function, the deployment pipeline executes the following sequence:

1. **Data Ingestion:** Safely captures user inputs via standard conversion.
2. **Transformation:** Normalizes the user's specific data points using the exact scaling boundaries learned during the training phase.
3. **Inference:** Computes the Euclidean distance against the 160 stored records, isolates the 5 closest profiles and tabulates the votes.
4. **Feedback:** Outputs the predicted grade alongside the confidence margin .

10. Scalability

Future Scalability

While this current iteration is a self-contained local application using data, tracking and updating the model is vital for real-world viability.

If deployed in a live university environment the system would rely on a continuous data pipeline rather than the generating data function. It would accumulate all the discrete data and give a final output .